

Unidad III: Variantes de los bucles **for** y funciones de orden superior

Isaac Alexander Morales Arévalo

Contents

3. Ejercicios con soluciones	1
Ejercicio 1	1
Ejercicio 2	2
Ejercicio 3	2
Ejercicio 4	2
Ejercicio 5	5
Ejercicio 6	6
Ejercicio 7	7
Ejercicio 8	7
Ejercicio 9	7
Ejercicio 10	7

```
library(dplyr)
library(purrr)
library(tibble)
```

3. Ejercicios con soluciones

Ejercicio 1

Usa la base de datos `mtcars`. Calcula, con un bucle `for`, la media de cada columna. Guarda los resultados en un vector numérico con nombres.

```
medias <- numeric(ncol(mtcars))
names(medias) <- names(mtcars)

for(i in 1:ncol(mtcars)) {
  medias[i] <- mean(mtcars[[i]])
}

medias
```

```
##      mpg      cyl      disp      hp      drat      wt      qsec
## 20.090625  6.187500 230.721875 146.687500  3.596563  3.217250 17.848750
##      vs      am      gear      carb
##  0.437500  0.406250  3.687500  2.812500
```

Ejercicio 2

Usa la base de datos `iris`. Con un bucle `for`, determina qué columnas son numéricas. Guarda el resultado en un vector lógico con nombres.

```
es_numerica <- logical(ncol(iris))
names(es_numerica) <- names(iris)

for(i in 1:ncol(iris)) {
  es_numerica[i] <- is.numeric(iris[[i]])
}

es_numerica
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width Species
##      TRUE      TRUE      TRUE      TRUE      FALSE
```

Ejercicio 3

Usa la base de datos `iris`. Con un bucle `for`, calcula la desviación estándar únicamente de las columnas numéricas.

```
desviaciones <- c()

for(i in 1:ncol(iris)){
  if(is.numeric(iris[[i]])){
    desviaciones[names(iris)[i]] <- sd(iris[[i]])
  }
}

desviaciones
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width
##  0.8280661    0.4358663    1.7652982    0.7622377
```

Ejercicio 4

Crea una función llamada `resumen_columnas()` que reciba un data frame y una función. La función debe aplicar la función dada a cada columna del data frame.

Prueba tu función con `mean`, `median` y `sd` usando `mtcars`.

```
resumen_columnas <- function(df, funcion) {

  resultados <- vector(mode = "list", length = ncol(df))
  names(resultados) <- names(df)
```

```

for(i in 1:ncol(df)) {
  resultados[[i]] <- funcion(df[[i]])
}

return(resultados)
}

# Pruebas con mtcars

resumen_columnas(mtcars, mean)

```

```

## $mpg
## [1] 20.09062
##
## $cyl
## [1] 6.1875
##
## $disp
## [1] 230.7219
##
## $hp
## [1] 146.6875
##
## $drat
## [1] 3.596563
##
## $wt
## [1] 3.21725
##
## $qsec
## [1] 17.84875
##
## $vs
## [1] 0.4375
##
## $am
## [1] 0.40625
##
## $gear
## [1] 3.6875
##
## $carb
## [1] 2.8125

```

```

resumen_columnas(mtcars, median)

```

```

## $mpg
## [1] 19.2
##
## $cyl
## [1] 6
##
## $disp

```

```
## [1] 196.3
##
## $hp
## [1] 123
##
## $drat
## [1] 3.695
##
## $wt
## [1] 3.325
##
## $qsec
## [1] 17.71
##
## $vs
## [1] 0
##
## $am
## [1] 0
##
## $gear
## [1] 4
##
## $carb
## [1] 2
```

```
resumen_columnas(mtcars, sd)
```

```
## $mpg
## [1] 6.026948
##
## $cyl
## [1] 1.785922
##
## $disp
## [1] 123.9387
##
## $hp
## [1] 68.56287
##
## $drat
## [1] 0.5346787
##
## $wt
## [1] 0.9784574
##
## $qsec
## [1] 1.786943
##
## $vs
## [1] 0.5040161
##
## $am
## [1] 0.4989909
```

```
##
## $gear
## [1] 0.7378041
##
## $carb
## [1] 1.6152
```

Ejercicio 5

Adapta la función anterior para que trabaje únicamente con columnas numéricas. Pruébala con `iris`.

```
resumen_numericas <- function(df, funcion) {

  resultados <- list()

  for(i in 1:ncol(df)) {

    if(is.numeric(df[[i]])) {
      resultados[[names(df)[i]]] <- funcion(df[[i]])
    }

  }

  return(resultados)
}

# Prueba con iris

resumen_numericas(iris, mean)
```

```
## $Sepal.Length
## [1] 5.843333
##
## $Sepal.Width
## [1] 3.057333
##
## $Petal.Length
## [1] 3.758
##
## $Petal.Width
## [1] 1.199333
```

```
resumen_numericas(iris, median)
```

```
## $Sepal.Length
## [1] 5.8
##
## $Sepal.Width
## [1] 3
##
## $Petal.Length
## [1] 4.35
```

```
##
## $Petal.Width
## [1] 1.3
```

```
resumen_numericas(iris, sd)
```

```
## $Sepal.Length
## [1] 0.8280661
##
## $Sepal.Width
## [1] 0.4358663
##
## $Petal.Length
## [1] 1.765298
##
## $Petal.Width
## [1] 0.7622377
```

Ejercicio 6

Genera 10 números aleatorios normales para cada una de las siguientes medias: -10, 0, 10 y 100. Usa un bucle for y guarda los resultados en una lista.

```
medias <- c(-10, 0, 10, 100)

resultados <- list()

for(i in 1:length(medias)) {
  resultados[[paste("media:", medias[i])]] <- rnorm(10, mean = medias[i])
}

resultados

## $'media: -10'
## [1] -10.655319 -9.016053 -10.033167 -10.347608 -11.739213 -9.309429
## [7] -9.786538 -9.697755 -10.998866 -10.135543
##
## $'media: 0'
## [1] 1.02556354 -0.42328741 -0.01533294 1.80305963 -0.63103439 0.54191926
## [7] -0.80873819 -0.74892110 0.03408302 0.91589104
##
## $'media: 10'
## [1] 11.899844 11.806865 8.248291 10.170928 11.183026 9.524146 9.473825
## [8] 9.987448 9.057160 8.476162
##
## $'media: 100'
## [1] 99.34851 99.72066 101.07805 100.92098 99.19090 99.94738 99.44901
## [8] 99.15928 98.53009 98.85972
```

Ejercicio 7

Reescribe el ejercicio anterior usando `map()`.

Ejercicio 8

Usa `map_dbl()` para calcular la media de cada columna de `mtcars`.

```
library(purrr)
```

```
medias_mtcars <- map_dbl(mtcars, mean)
```

```
medias_mtcars
```

```
##      mpg      cyl    disp      hp      drat      wt      qsec
## 20.090625  6.187500 230.721875 146.687500  3.596563  3.217250 17.848750
##      vs      am      gear      carb
##  0.437500  0.406250  3.687500  2.812500
```

Ejercicio 9

Usa `map_chr()` para obtener la clase de cada columna de `iris`.

```
clases_iris <- map_chr(iris, class)
```

```
clases_iris
```

```
## Sepal.Length Sepal.Width Petal.Length Petal.Width Species
##      "numeric"      "numeric"      "numeric"      "numeric"      "factor"
```

Ejercicio 10

Usa un bucle `while` para simular lanzamientos de una moneda hasta obtener dos caras seguidas. El resultado final debe ser el número de lanzamientos necesarios.

```
set.seed(123)
```

```
lanzamientos <- 0
```

```
caras_seguidas <- 0
```

```
while(caras_seguidas < 2) {
```

```
  moneda <- sample(c("cara", "cruz"), size = 1)
```

```
  lanzamientos <- lanzamientos + 1
```

```
  cat("Lanzamiento", lanzamientos, ":", moneda, "\n")
```

```
  if(moneda == "cara") {
```

```
    caras_seguidas <- caras_seguidas + 1
```

```
} else {  
  caras_seguidas <- 0  
}  
  
}
```

```
## Lanzamiento 1 : cara  
## Lanzamiento 2 : cara
```

```
cat("\nSe necesitaron", lanzamientos,  
    "lanzamientos para obtener dos caras seguidas.\n")
```

```
##  
## Se necesitaron 2 lanzamientos para obtener dos caras seguidas.
```